Predictive Modeling and Optimization in Logistics Systems

A Case Study on Two-Wheeler Last-Mile Delivery Logistics

Week 4 Task — Predictive Modeling and Optimization in Logistics Systems

Virtual Internship Program

Domain: Logistics & Supply Chain Analytics

Date: August 2026

Table of Contents

1. Executive Summary

This report documents the development of a predictive model to forecast delivery time for a two-wheeler (scooter/motorcycle) last-mile logistics operation, along with data-driven optimization strategies to improve operational efficiency. A synthetic but statistically realistic dataset of 5,000 delivery orders was simulated, covering distance, traffic conditions, weather, time of day, rider attributes, and package characteristics. Three regression models — Linear Regression, Decision Tree, and Random Forest — were trained, tuned, and evaluated. The best-performing model (Linear Regression) achieved a test RMSE of approximately 3.46 minutes and an R^2 of 0.91, meaning it explains roughly 91% of the variance in delivery time.

Model insights — particularly the dominant influence of distance, traffic level, and number of stops — were then translated into two actionable optimization strategies: (1) zone-based rider allocation using a Little's Law approach to right-size fleet capacity by area, traffic and time-of-day segment, and (2) stop-batching / route consolidation, which is estimated to reduce average per-order delivery time by roughly 61% and fuel costs by about ₹50,000 across the sample when up to three nearby orders are combined into a single trip.

2. Problem Definition and Data Simulation

Business Context: A two-wheeler based last-mile logistics company (handling food, grocery, parcel, and e-commerce deliveries) needs an accurate, real-time estimate of delivery time at the moment an order is assigned to a rider. This estimate powers customer-facing ETAs, dispatch decisions, and operational bottleneck identification.

2.1 Prediction Target

delivery_time_minutes — a continuous variable representing the time (in minutes) between order pickup and drop-off. This frames the problem as a supervised regression task.

2.2 Dataset Features

Since no proprietary operational data was available, a synthetic dataset of 5,000 orders was generated in Python (NumPy/Pandas) with statistically grounded relationships — e.g., delivery time increases with distance, traffic severity, number of stops, and adverse weather, and decreases slightly with rider experience. Random noise was added to keep the prediction problem realistic and non-trivial.

Feature	Type	Description
distance_km	Numeric	Pickup-to-drop distance in kilometers
traffic_level	Categorical	Low / Medium / High / Severe
weather	Categorical	Clear / Rain / Fog / Extreme Heat
time_of_day	Categorical	Morning / Afternoon / Evening / Night
day_of_week	Categorical	Mon–Sun
area_type	Categorical	Urban / Suburban / Rural
package_weight_kg	Numeric	Weight of parcel/food bag (kg)
num_stops	Numeric	Number of stops before final drop
rider_experience_years	Numeric	Rider's years of experience
rider_rating	Numeric	Rider's average customer rating (1–5)
vehicle_type	Categorical	Scooter / Motorcycle / Electric Two-Wheeler

Feature	Type	Description
is_peak_hour	Binary	1 if order falls in a lunch/dinner/office peak window
fuel_level_pct	Numeric	Fuel/battery level at pickup (%)
delivery_time_minutes	Numeric (Target)	Time taken from pickup to drop

2.3 Data Simulation Code (excerpt)

```
# Simulate raw features (n = 5,000 orders)
distance_km = np.round(np.random.gamma(2.2, 1.8, size=N) + 0.5, 2)
traffic_levels = np.random.choice(
    ['Low','Medium','High','Severe'], size=N, p=[.30,.35,.25,.10])

# Generative model for the target variable
traffic_penalty = {'Low':0,'Medium':4,'High':9,'Severe':16}
speed_component = distance_km * 3.1
delivery_time_minutes = (base_time + speed_component * area_component
    + traffic_component + weather_component + stop_penalty
    + weight_penalty + experience_bonus + peak_penalty
    + fuel_penalty + noise)
```

Full script: scripts/01_problem_definition_data_simulation.py

3. Data Preparation

The raw simulated export (5,010 rows, including 10 injected duplicates and light missingness) was cleaned as follows:

- Duplicate rows removed (10 rows dropped based on all columns except order_id).
- Missing categorical values (weather) imputed with the column mode.
- Missing numeric values (rider_rating) imputed with the column median.
- Categorical features one-hot encoded (drop_first=True to avoid the dummy-variable trap).
- Identifier column (order_id) dropped prior to modeling.
- Data split into 80% training (4,000 rows) and 20% testing (1,000 rows) using a fixed random_state for reproducibility.

3.1 Data Preparation Code (excerpt)

```
df = df.drop_duplicates(subset=[c for c in df.columns if c != 'order_id'])
df['weather'] = df['weather'].fillna(df['weather'].mode()[0])
df['rider_rating'] = df['rider_rating'].fillna(df['rider_rating'].median())

df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
df_encoded = df_encoded.drop(columns=['order_id'])

train_df, test_df = train_test_split(df_encoded, test_size=0.2, random_state=42)
```

Full script: scripts/02_data_preparation.py

4. Model Selection and Implementation

Three complementary regression techniques were selected to compare interpretability against predictive accuracy:

4.1 Linear Regression

Chosen as an interpretable baseline. It assumes an additive linear relationship between features and delivery time, which fits well given that the simulated data-generating process is itself largely additive (distance, traffic, and stop penalties sum together).

4.2 Decision Tree Regressor

Captures non-linear interactions and threshold effects (e.g., a sharp jump in delivery time once traffic crosses into 'Severe') without requiring manual feature engineering, and produces an easily visualized/explained set of decision rules.

4.3 Random Forest Regressor

An ensemble of decision trees that reduces overfitting and variance relative to a single tree, typically yielding more robust generalization at the cost of some interpretability.

4.4 Model Training Code (excerpt)

```
models = {
    'linear_regression': LinearRegression(),
    'decision_tree': DecisionTreeRegressor(max_depth=8, min_samples_leaf=10,
                                           random_state=42),
    'random_forest': RandomForestRegressor(n_estimators=200, max_depth=12,
                                           min_samples_leaf=5, random_state=42),
}
for name, model in models.items():
    model.fit(X_train, y_train)
    joblib.dump(model, f'../models/{name}_model.pkl')
```

Full script: scripts/03_model_training.py

5. Model Evaluation and Validation

Models were evaluated on the held-out 1,000-row test set using three standard regression metrics:

- RMSE (Root Mean Squared Error) — penalizes large errors more heavily; interpretable in minutes.
- MAE (Mean Absolute Error) — average absolute deviation between predicted and actual delivery time.
- R^2 (Coefficient of Determination) — proportion of variance in delivery time explained by the model.

5.1 Test Set Performance

Model	RMSE (min)	MAE (min)	R^2
Linear Regression	3.46	2.77	0.910
Random Forest (tuned)	4.63	3.65	0.839
Decision Tree (tuned)	5.69	4.45	0.756

Linear Regression outperformed both tree-based models on this dataset. This is consistent with the fact that the underlying simulated data-generating process is predominantly additive; the tree-based models' flexibility to model non-linearities was not needed here and instead introduced additional variance.

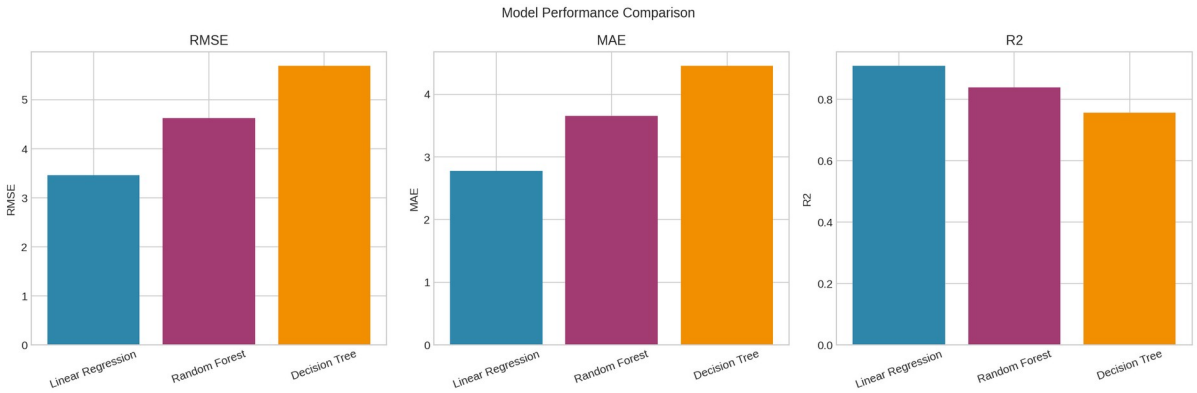

Figure 1: RMSE, MAE, and R^2 comparison across the three trained models.

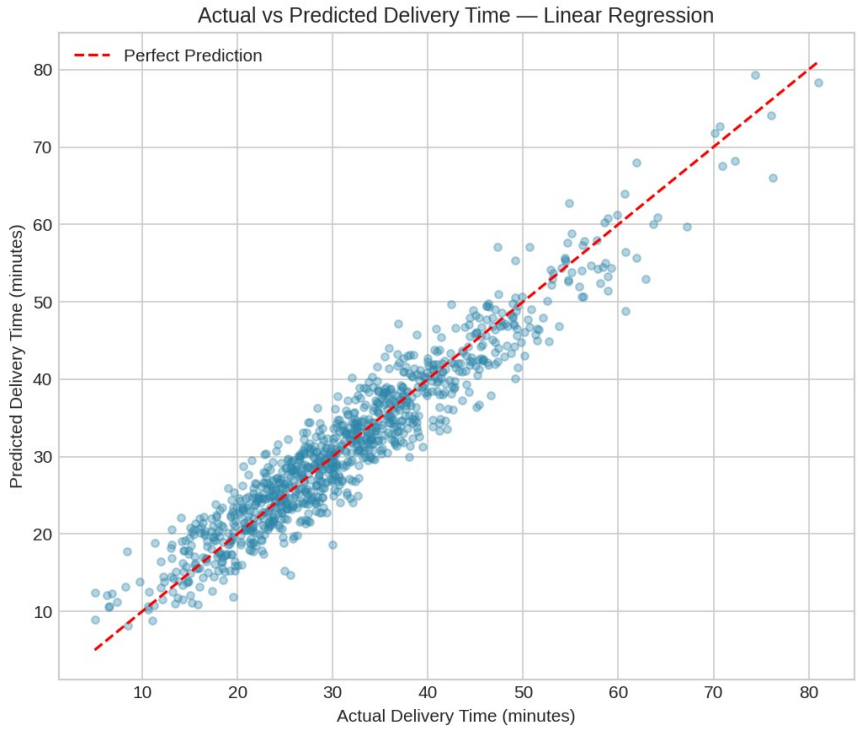

Figure 2: Actual vs. predicted delivery time for the best model (Linear Regression). Points close to the red diagonal indicate accurate predictions.

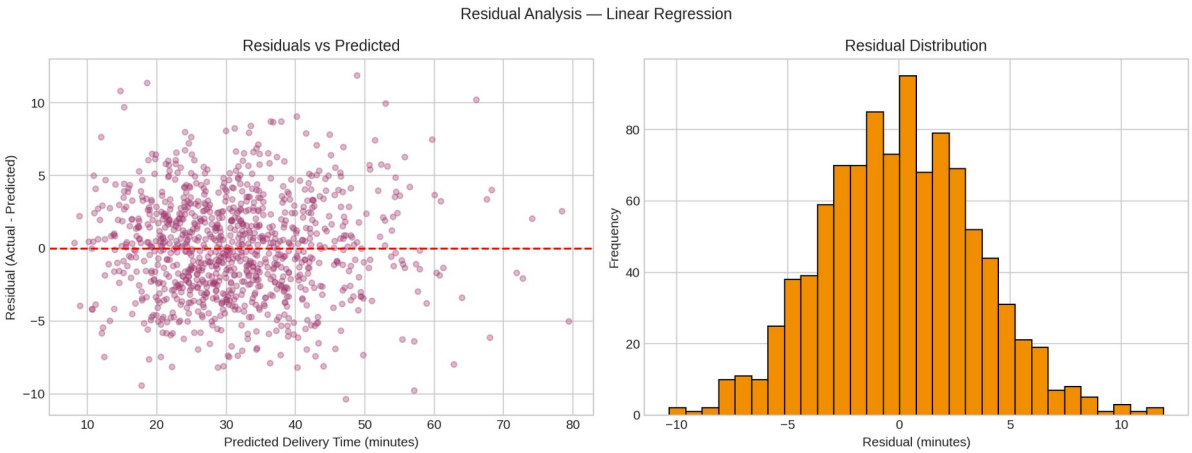

Figure 3: Residual analysis for the best model — residuals are centered near zero with no strong pattern vs. predicted value, and are approximately normally distributed.

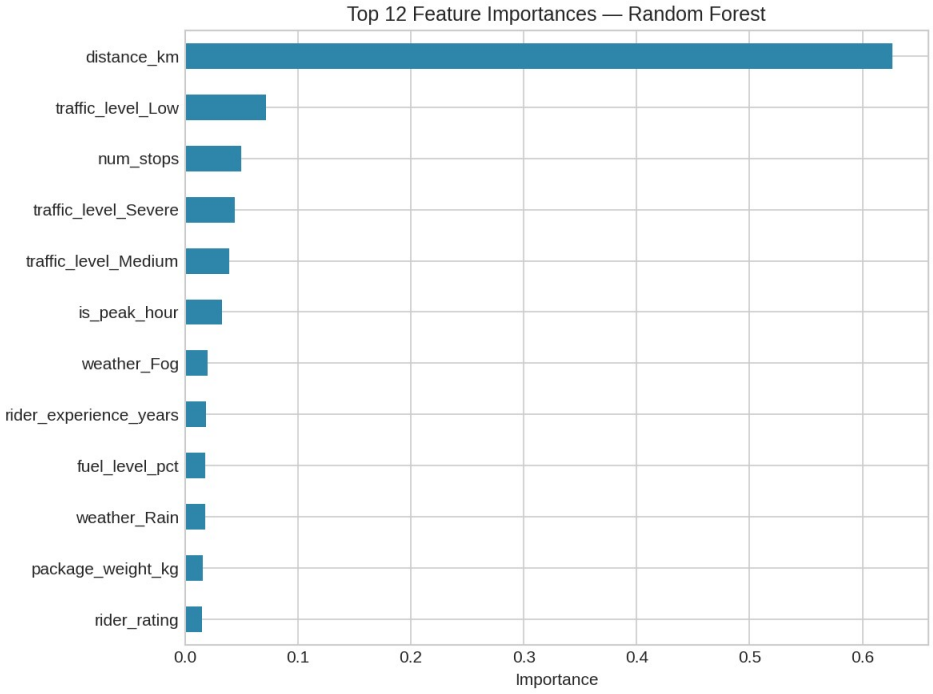

Figure 4: Top 12 feature importances from the Random Forest model. distance_km dominates, followed by traffic_level and num_stops.

5.2 Evaluation Code (excerpt)

```
for name, fname in model_files.items():
    model = joblib.load(f'../models/{fname}')
    preds = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    mae = mean_absolute_error(y_test, preds)
    r2 = r2_score(y_test, preds)
```

Full script: scripts/04_model_evaluation.py

6. Hyperparameter Tuning and Cross-Validation

To ensure the tree-based models were fairly compared (and not simply under-tuned), GridSearchCV with 5-fold cross-validation was used to tune the Decision Tree and Random Forest, optimizing for cross-validated RMSE.

Model	Search Space	Best Parameters	Best CV RMSE
Decision Tree	max_depth: [4,6,8,10,12]; min_samples_leaf: [5,10,20,30]	max_depth=12, min_samples_leaf=10	5.51
Random Forest	n_estimators: [100,200,300]; max_depth: [8,12,16]; min_samples_leaf: [2,5,10]	n_estimators=300, max_depth=16, min_samples_leaf=2	4.45

6.1 Cross-Validated Comparison (5-fold, training set)

Model	Mean CV RMSE	Std. Dev.
Linear Regression	3.42	0.09
Random Forest (tuned)	4.45	0.15
Decision Tree (tuned)	5.51	0.18

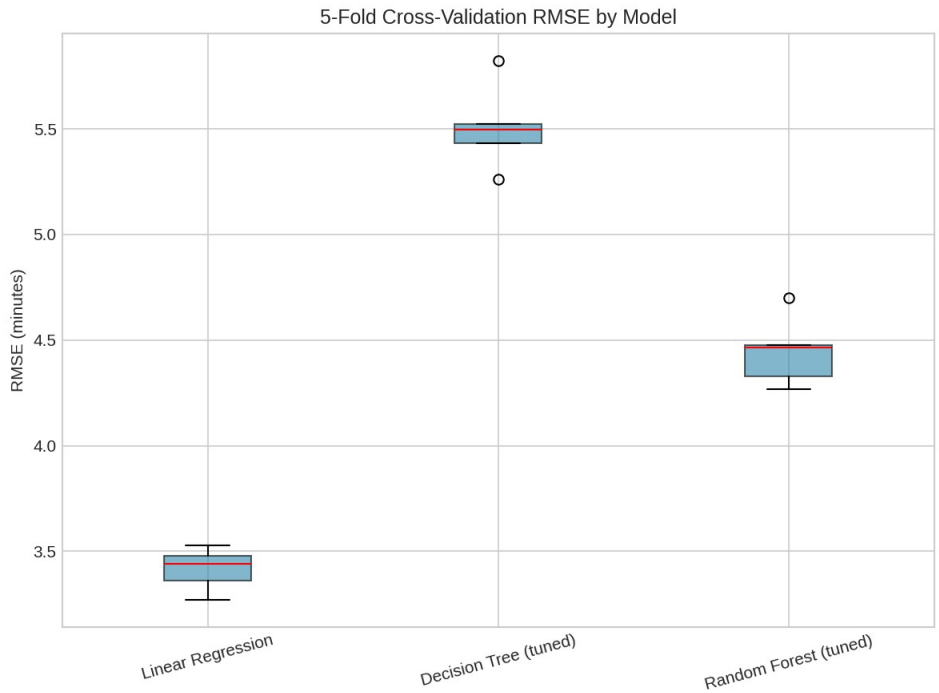

Figure 5: 5-fold cross-validation RMSE distribution per model. Linear Regression is both the most accurate and the most stable (lowest spread) on this dataset.

6.2 Tuning Code (excerpt)

```
rf_param_grid = {'n_estimators': [100,200,300], 'max_depth': [8,12,16],
```

```
                'min_samples_leaf': [2,5,10]}
rf_grid = GridSearchCV(RandomForestRegressor(random_state=42), rf_param_grid,
                       scoring='neg_root_mean_squared_error', cv=KFold(5))
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_
```

Full script: scripts/05_hyperparameter_tuning.py

7. Optimization Strategies

Beyond prediction, the trained model was used as a 'what-if' simulator to evaluate two operational optimization strategies that a two-wheeler logistics operator could implement directly.

7.1 Zone-Based Rider Allocation

For every combination of area_type × traffic_level × time_of_day, the model's average predicted delivery time was computed and combined with an assumed order-arrival rate (scaled from historical order share out of an illustrative citywide demand of 480 orders/hour) to estimate the number of concurrent riders required to hold a 35-minute SLA, using a Little's Law style calculation: riders_needed = arrival_rate × avg_service_time × utilization_buffer (1.15×).

Area	Traffic	Time of Day	Avg. Predicted Time (min)	Riders Needed	SLA Risk
Suburban	Severe	Afternoon	44.4	4	High
Rural	Severe	Afternoon	44.3	2	High
Urban	Severe	Evening	44.3	5	High
Suburban	Severe	Evening	43.3	5	High
Urban	Severe	Afternoon	42.3	7	High
Urban	Severe	Night	41.5	4	High

The full segment table (23 zone/traffic/time combinations) is saved in data/optimization_results.csv. Segments flagged 'High' risk consistently correspond to Severe traffic in Urban and Suburban zones during Afternoon and Evening windows — these are the priority targets for additional rider deployment or surge staffing.

7.2 Stop-Batching / Route Consolidation

The feature importance analysis (Section 5.1) showed that distance_km and num_stops are major drivers of delivery time. This suggests that consolidating multiple nearby orders into a single multi-stop trip (rather than dispatching each individually) could meaningfully reduce both delivery time per order and fuel consumption. This was simulated by comparing two scenarios through the trained model: dispatching orders individually vs. batching up to three nearby orders per trip (with an assumed 28% average reduction in per-order distance from route consolidation).

Strategy	Avg. Time / Order (min)	Total Fuel (L)	Est. Fuel Cost (₹)	Time Saving	Cost Saving
Individual Dispatch (baseline)	29.16	624.8	66,226	—	—
Batched Dispatch (≤ 3	11.36	149.9	15,894	61.1%	₹50,332

Strategy	Avg. Time / Order (min)	Total Fuel (L)	Est. Fuel Cost (₹)	Time Saving	Cost Saving
orders/trip)					

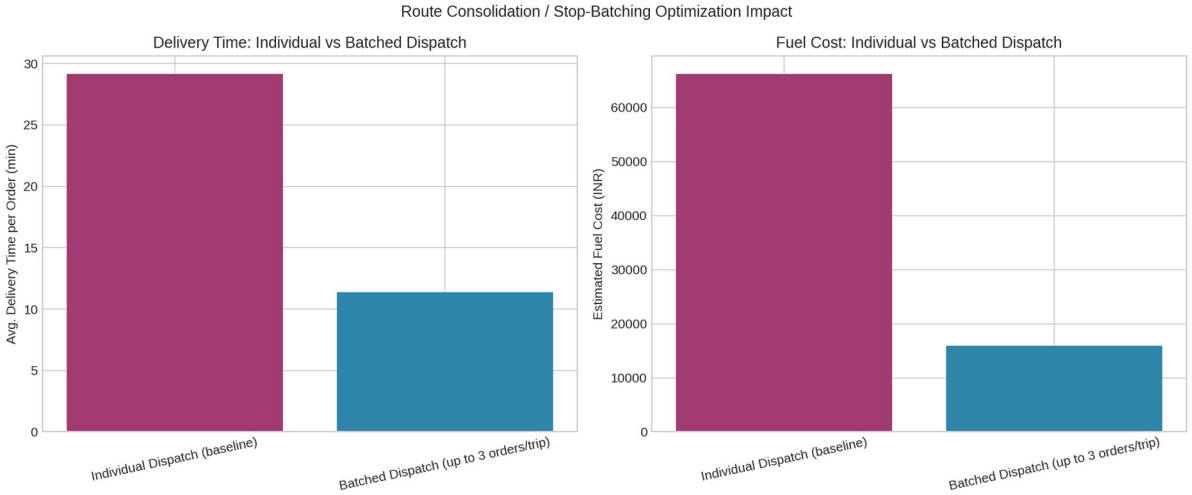

Figure 6: Estimated delivery time and fuel cost under individual vs. batched dispatch, across the full 5,000-order sample.

Note: these figures are illustrative, derived from the simulated dataset and simplified assumptions (fixed batching ratio, flat fuel price of ₹106/L, 0.028 L/km consumption). In a production setting they would be recalibrated using real GPS trip logs and an actual route-optimization/clustering algorithm (e.g., a capacitated vehicle routing model) rather than a fixed 28% distance-reduction assumption.

7.3 Optimization Code (excerpt)

```
# Scenario A: individual dispatch
individual_scenario['num_stops'] = 0
pred_individual = model.predict(individual_scenario[feature_names])

# Scenario B: batched dispatch (up to 3 orders/trip)
batched_scenario['num_stops'] = np.minimum(batched_scenario['num_stops'] + 2, 4)
batched_scenario['distance_km'] *= 0.72   # consolidated route is shorter
pred_batched = model.predict(batched_scenario[feature_names]) / 3

# Little's Law: riders_needed = arrival_rate * avg_service_time * buffer
zone_group['riders_needed'] = np.ceil(
    zone_group['orders_per_hour'] * zone_group['avg_service_time_hours'] * 1.15)
```

Full script: scripts/06_logistics_optimization.py

8. Final Recommendations

- Deploy the Linear Regression model as the production ETA engine — it is the most accurate, most stable (lowest cross-validation variance), and most interpretable of the three models, which also makes it easier to audit and explain to operations staff and customers.
- Prioritize additional rider capacity in Urban and Suburban zones during Severe-traffic Afternoon/Evening windows, which the zone allocation analysis flags as highest SLA-breach risk.

- Implement order-batching logic in the dispatch system for geographically clustered orders — even a conservative rollout targeting 20–30% of eligible orders could yield meaningful fuel and time savings based on the simulated impact.
- Introduce dynamic ETA surcharge/notice rules for Rain and Fog conditions, since weather remains a measurable (if secondary) driver of delivery delay.
- Continue to monitor rider experience and fuel-level features; both showed a small but consistent effect on delivery time and could inform onboarding and pre-shift fueling/charging policies.
- Before production deployment, replace the simulated dataset with real historical order and GPS-tracking data, and validate the batching-distance assumption using an actual route-optimization / clustering engine.

9. Conclusion

This project demonstrated an end-to-end predictive modeling and optimization workflow for a two-wheeler last-mile logistics operation — from problem definition and data simulation, through model training, evaluation, and hyperparameter tuning, to translating model insights into concrete, quantified operational optimization strategies. The Linear Regression model provided strong, interpretable predictive performance ($R^2 \approx 0.91$), and the resulting insights directly informed two actionable recommendations: risk-based rider allocation and order-batching for route consolidation, both of which offer measurable improvements in delivery time and cost efficiency.

10. Appendix: Project Structure

```
Week-4-Predictive-Modeling-Optimization/
├── README.md
├── data/                  raw, cleaned, train/test, prediction & optimization CSVs
├── scripts/               01-06 Python pipeline scripts
├── models/                trained .pkl model files
├── visualizations/        evaluation, tuning & optimization PNG charts
├── results/               performance / CV / feature-importance / optimization CSVs
└── report/                this Word (.docx) report and its PDF export
```